

Workflow and architecture patterns for working together 2: Architectures

Jacob Archambault

May 26, 2026

Outline

1 Current challenges and their canonical solutions

Outline

- ① Current challenges and their canonical solutions
- ② Branching: theory and statistics
 - Theory
 - Statistics
 - Velocity
 - Safety

Outline

- 1 Current challenges and their canonical solutions
- 2 Branching: theory and statistics
 - Theory
 - Statistics
 - Velocity
 - Safety
- 3 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines

Outline

- 1 Current challenges and their canonical solutions
- 2 Branching: theory and statistics
 - Theory
 - Statistics
 - Velocity
 - Safety
- 3 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 4 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

Challenges

- multiple teams working in the same repository

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks
- coupling of all services at the database layer

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks
- coupling of all services at the database layer
 - high storage costs

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks
- coupling of all services at the database layer
 - high storage costs
 - single point of failure risk

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks
- coupling of all services at the database layer
 - high storage costs
 - single point of failure risk
 - risk of cross-project interference

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks
- coupling of all services at the database layer
 - high storage costs
 - single point of failure risk
 - risk of cross-project interference

Canonical solutions to these challenges

- multiple teams $\rightarrow$ inverse Conway maneuver

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow
- database coupling → Bezos API mandate/“Microservices”

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow
- database coupling → Bezos API mandate/“Microservices”

Outline

- 1 Current challenges and their canonical solutions
- 2 Branching: theory and statistics
 - Theory
 - Statistics
 - Velocity
 - Safety
- 3 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 4 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

A naive probability theory for branch incompatibility: isolated commits

- $C = \{a, b, c\}$

A naive probability theory for branch incompatibility: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$

A naive probability theory for branch incompatibility: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\text{no conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$

A naive probability theory for branch incompatibility: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\text{no conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(\text{no conflicts}) = \frac{1}{2^n}$ where $|C| = n$

A naive probability theory for branch incompatibility: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\text{no conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(\text{no conflicts}) = \frac{1}{2^n}$ where $|C| = n$
 - $P(\text{no conflicts}) < 10\%$ where $n \geq 4$

A naive probability theory for branch incompatibility: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\text{no conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(\text{no conflicts}) = \frac{1}{2^n}$ where $|C| = n$
 - $P(\text{no conflicts}) < 10\%$ where $n \geq 4$
 - $P(\text{no conflicts}) < 1\%$ where $n \geq 7$

A naive probability theory for branch incompatibility: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\text{no conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(\text{no conflicts}) = \frac{1}{2^n}$ where $|C| = n$
 - $P(\text{no conflicts}) < 10\%$ where $n \geq 4$
 - $P(\text{no conflicts}) < 1\%$ where $n \geq 7$

A naive probability theory for branch incompatibility: sets of commits

- $A = A_1 \sqcup A_2 \sqcup \dots \sqcup A_i$

A naive probability theory for branch incompatibility: sets of commits

- $A = A_1 \sqcup A_2 \sqcup \dots \sqcup A_i$
- e.g., $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$

A naive probability theory for branch incompatibility: sets of commits

- $A = A_1 \sqcup A_2 \sqcup \dots \sqcup A_i$
- e.g., $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$

- a simplified theory probability theory for incompatible changes -
- Cartesian products commits across n branches over time -
- Cartesian explosion, - intra vs. inter team communication

Outline

- 1 Current challenges and their canonical solutions
- 2 Branching: theory and statistics
 - Theory
 - Statistics
 - Velocity
 - Safety
- 3 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 4 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

- compare total commits across team branch vs. others - compare non-merge commits to non-PR merge commits across team branch vs. others (this represents percentage of value-adding work to maintenance work) (`git log --oneline --merges | grep -iv "merge pull request" | wc -l`)
- compare time to merge across SpecFlow branches vs. others
- compare passing to failing pipeline runs across team branch vs.

others, for both simulation pipeline and unit test pipeline

Outline

- 1 Current challenges and their canonical solutions
- 2 Branching: theory and statistics
 - Theory
 - Statistics
 - Velocity
 - Safety
- 3 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 4 Architecture
 - Techniques for integrating partially complete work
 - Clean code

Team composition

- all teams working in same git repository are in the same daily scrum

Team composition

- all teams working in same git repository are in the same daily scrum
- each team in the above sense has its own Microsoft Teams chat with only its own contributors in it

Team composition

- all teams working in same git repository are in the same daily scrum
- each team in the above sense has its own Microsoft Teams chat with only its own contributors in it
- these teams are *stream-aligned*, i.e., they focus on a particular business area (in our case, either a transaction type or a closely related set of transaction types)

Team composition

- all teams working in same git repository are in the same daily scrum
- each team in the above sense has its own Microsoft Teams chat with only its own contributors in it
- these teams are *stream-aligned*, i.e., they focus on a particular business area (in our case, either a transaction type or a closely related set of transaction types)
- these teams are *autonomous*, i.e., they can move work forward without requiring outside assistance from another team

Team composition

- all teams working in same git repository are in the same daily scrum
- each team in the above sense has its own Microsoft Teams chat with only its own contributors in it
- these teams are *stream-aligned*, i.e., they focus on a particular business area (in our case, either a transaction type or a closely related set of transaction types)
- these teams are *autonomous*, i.e., they can move work forward without requiring outside assistance from another team

Team composition: enabling teams

- Strategy will be rolled out first for claims, then reimbursements, then other transactions

Team composition: enabling teams

- Strategy will be rolled out first for claims, then reimbursements, then other transactions
- the 837 COB team will serve as an enabling team for the new branching strategy

Team composition: enabling teams

- Strategy will be rolled out first for claims, then reimbursements, then other transactions
- the 837 COB team will serve as an enabling team for the new branching strategy

Outline

- 1 Current challenges and their canonical solutions
- 2 Branching: theory and statistics
 - Theory
 - Statistics
 - Velocity
 - Safety
- 3 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 4 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

Branching and committing

- all commit work done by a given team, by both developers and testers, is committed directly to the same shared branch

Branching and committing

- all commit work done by a given team, by both developers and testers, is committed directly to the same shared branch
- every commit is pushed to origin after passing local testing

Branching and committing

- all commit work done by a given team, by both developers and testers, is committed directly to the same shared branch
- every commit is pushed to origin after passing local testing
- every git commit message begins with a Jira ticket number

Branching and committing

- all commit work done by a given team, by both developers and testers, is committed directly to the same shared branch
- every commit is pushed to origin after passing local testing
- every git commit message begins with a Jira ticket number

Outline

- 1 Current challenges and their canonical solutions
- 2 Branching: theory and statistics
 - Theory
 - Statistics
 - Velocity
 - Safety
- 3 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 4 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

- Every change is tested by the test suite locally before committing

- tests are written against new code as that code is being written

Outline

- 1 Current challenges and their canonical solutions
- 2 Branching: theory and statistics
 - Theory
 - Statistics
 - Velocity
 - Safety
- 3 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 4 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

- daily PRs - stacked PRs

Outline

- 1 Current challenges and their canonical solutions
- 2 Branching: theory and statistics
 - Theory
 - Statistics
 - Velocity
 - Safety
- 3 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 4 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

- pipeline is definitive for deployment - daily stale branch job

Outline

- 1 Current challenges and their canonical solutions
- 2 Branching: theory and statistics
 - Theory
 - Statistics
 - Velocity
 - Safety
- 3 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 4 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

Outline

- 1 Current challenges and their canonical solutions
- 2 Branching: theory and statistics
 - Theory
 - Statistics
 - Velocity
 - Safety
- 3 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 4 **Architecture**
 - Techniques for integrating partially complete work
 - **Clean code**
 - On the way to Microservices

Outline

- 1 Current challenges and their canonical solutions
- 2 Branching: theory and statistics
 - Theory
 - Statistics
 - Velocity
 - Safety
- 3 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 4 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices